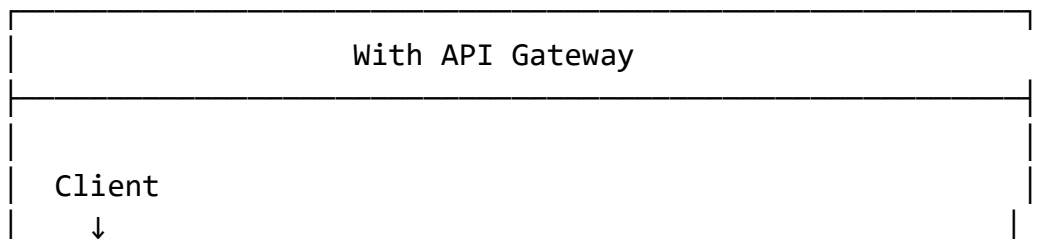
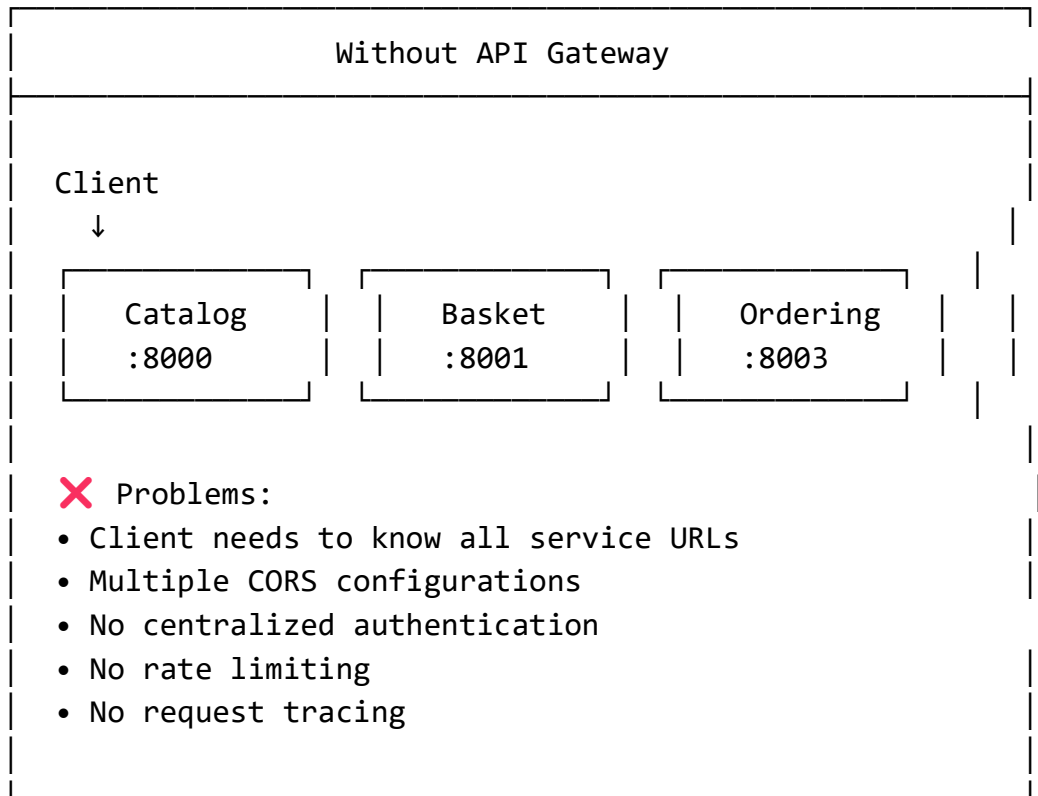


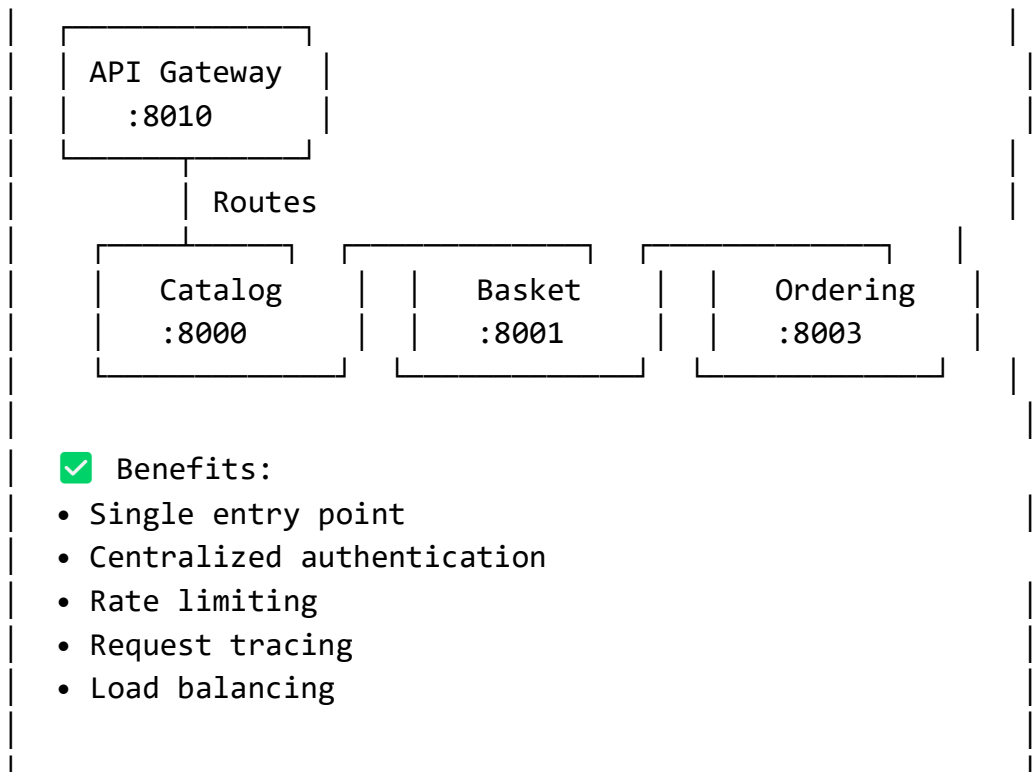
API Gateway & Rate Limiting - Complete Guide Using Your Project

Part 1: What is API Gateway?

Definition

An API Gateway is a **single entry point** for all your microservices. It acts like a **receptionist** that directs requests to the right service.





Part 2: Why API Gateway is Used in Your Project

Your Current Architecture

You ARE using API Gateway with Ocelot library.

File: ApiGateway.csproj (Line 17)

```
<PackageReference Include="Ocelot" Version="24.1.0" />
```

Problems Your API Gateway Solves

1. Single Entry Point

Instead of:

- <http://localhost:8000/api/v1/Catalog/GetAllProducts>
- <http://localhost:8001/api/v1/Basket/john>
- <http://localhost:8003/api/v1/Order/john>

Client uses:

- <http://localhost:8010/Catalog/GetAllProducts>
- <http://localhost:8010/Basket/john>
- <http://localhost:8010/Order/john>

2. Centralized Authentication

```
// Program.cs (Lines 42-55)
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {
        options.TokenValidationParameters = new
TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = issuer,
            ValidAudience = audience,
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(key))
        };
    });
```

3. Request Tracing

```
// CorrelationIdMiddleware.cs (Lines 24-32)
if (!HttpContext.Request.Headers.TryGetValue(CorrelationIdHeader, out
var correlationId))
{
    correlationId = Guid.NewGuid().ToString();
    HttpContext.Request.Headers[CorrelationIdHeader] = correlationId;
}
HttpContext.Response.Headers[CorrelationIdHeader] = correlationId;
```

4. Service Discovery API Gateway knows where all services are located:

- Catalog: host.docker.internal:8000
- Basket: host.docker.internal:8001
- Ordering: host.docker.internal:8003
- Identity: host.docker.internal:8005

Part 3: How You Implemented API Gateway

Step 1: Install Ocelot Packages

File: ApiGateway.csproj (Lines 17-19)

```
<PackageReference Include="Ocelot" Version="24.1.0" />
<PackageReference Include="Ocelot.Cache.CacheManager" Version="24.1.0" />
<PackageReference
  Include="Microsoft.AspNetCore.Authentication.JwtBearer"
  Version="10.0.2" />
```

Step 2: Configure Ocelot

File: Program.cs (Lines 24-27, 59-60)

```
builder.Configuration
    .SetBasePath(Directory.GetCurrentDirectory())
    .AddJsonFile($"ocelot.{env}.json", optional: false,
reloadOnChange: true)
    .AddEnvironmentVariables();

builder.Services.AddOcelot(builder.Configuration)
    .AddCacheManager(x => x.WithDictionaryHandle());
```

Step 3: Define Routes

File: ocelot.Development.json (Lines 2-251)

Example Route for Catalog Service:

```
{
  "DownstreamPathTemplate": "/api/v1/Catalog/GetAllProducts",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [
    {
      "Host": "host.docker.internal",
      "Port": 8000
    }
  ],
  "UpstreamPathTemplate": "/Catalog/GetAllProducts",
  "UpstreamHttpMethod": [ "GET" ],
  "AddHeadersToRequest": {
    "x-correlation-id": "{X-Correlation-Id}"
  }
}
```

Route Breakdown:

- **UpstreamPathTemplate:** What client calls (/Catalog/GetAllProducts)
- **DownstreamPathTemplate:** What service receives (/api/v1/Catalog/GetAllProducts)
- **DownstreamHostAndPorts:** Where service is located (host.docker.internal:8000)
- **AddHeadersToRequest:** Add correlation ID for tracing

Step 4: Use Ocelot Middleware

File: Program.cs (Line 95)

```
await app.UseOcelot();
```

Part 4: Rate Limiting in Your Project

Current Status: Rate Limiting is Configured but DISABLED

Evidence:

```
// ocelot.Development.json (Lines 14-20, 171-177, 241-247)
// "RateLimitOptions": {
//   "EnableRateLimiting": true,
//   "Period": "10s",
//   "Limit": 5,
//   "ClientWhitelist": [],
//   "EnableHeaders": true
// },
```

Program.cs (Line 61):

```
// .AddRateLimiting(); <--- Keep this commented or removed
```

What Rate Limiting Does

Without Rate Limiting:

```
Client → API Gateway → Service
Client → API Gateway → Service
Client → API Gateway → Service
Client → API Gateway → Service
Client → API Gateway → Service
Client → API Gateway → Service
Client → API Gateway → Service ← Unlimited requests!
```

With Rate Limiting:

```
Client → API Gateway → Service ← 1st request (allowed)
Client → API Gateway → Service ← 2nd request (allowed)
Client → API Gateway → Service ← 3rd request (allowed)
Client → API Gateway → Service ← 4th request (allowed)
Client → API Gateway → Service ← 5th request (allowed)
```

Client → API Gateway → Service ← 6th request (BLOCKED! 429 Too Many Requests)

How to Enable Rate Limiting

Step 1: Install Rate Limiting Package

```
dotnet add package Ocelot.Provider.Polly
```

Step 2: Update Program.cs

```
// Remove comment from Line 61
builder.Services.AddOcelot(builder.Configuration)
    .AddCacheManager(x => x.WithDictionaryHandle())
    .AddPolly(); // Add this line
```

Step 3: Uncomment Rate Limiting in ocelot.json

For specific routes:

```
{
  "DownstreamPathTemplate": "/api/v1/Basket/Checkout",
  "DownstreamScheme": "http",
  "DownstreamHostAndPorts": [
    {
      "Host": "host.docker.internal",
      "Port": 8001
    }
  ],
  "UpstreamPathTemplate": "/Basket/Checkout",
  "UpstreamHttpMethod": [ "POST" ],
  "RateLimitOptions": {
    "EnableRateLimiting": true,
    "Period": "10s",
    "Limit": 5,
    "ClientWhitelist": [],
```

```
    "EnableHeaders": true
  },
  "AuthenticationOptions": {
    "AuthenticationProviderKey": "Bearer"
  }
}
```

Global rate limiting:

```
"GlobalConfiguration": {
  "BaseUrl": "http://localhost:8010",
  "RateLimitOptions": {
    "DisableRateLimitHeaders": false,
    "QuotaExceededMessage": "Too many requests! Please try again later.",
    "HttpStatusCode": 429,
    "ClientIdHeader": "Authorization"
  }
}
```

Part 5: Step-by-Step Implementation Guide

Step 1: Create API Gateway Project

```
dotnet new webapi -n ApiGateway
cd ApiGateway
```

Step 2: Install Ocelot Packages

```
dotnet add package Ocelot
dotnet add package Ocelot.Cache.CacheManager
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package Ocelot.Provider.Polly
```

Step 3: Create ocelot.json Configuration

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/v1/Catalog/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 8000
        }
      ],
      "UpstreamPathTemplate": "/Catalog/{everything}",
      "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ],
      "RateLimitOptions": {
        "EnableRateLimiting": true,
        "Period": "10s",
        "Limit": 10,
        "EnableHeaders": true
      }
    }
  ],
  "GlobalConfiguration": {
    "BaseUrl": "http://localhost:8010"
  }
}
```

Step 4: Configure Program.cs

```
using Ocelot.DependencyInjection;
using Ocelot.Middleware;
using Ocelot.Cache.CacheManager;
using Ocelot.Provider.Polly;

var builder = WebApplication.CreateBuilder(args);

// Load Ocelot configuration
```

```

builder.Configuration
    .SetBasePath(Directory.GetCurrentDirectory())
    .AddJsonFile("ocelot.json", optional: false, reloadOnChange: true)
    .AddEnvironmentVariables();

// Add authentication
var jwtSettings = builder.Configuration.GetSection("Jwt");
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {
        options.TokenValidationParameters = new
TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtSettings["Issuer"],
            ValidAudience = jwtSettings["Audience"],
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings["Key"]))
        };
    });

// Add Ocelot services
builder.Services.AddOcelot()
    .AddCacheManager(x => x.WithDictionaryHandle())
    .AddPolly();

var app = builder.Build();

app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

await app.UseOcelot();
app.Run();

```

Step 5: Add Custom Middleware (Optional)

CorrelationIdMiddleware.cs:

```
public class CorrelationIdMiddleware
{
    private const string CorrelationIdHeader = "x-correlation-id";
    private readonly RequestDelegate _next;

    public CorrelationIdMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext httpContext)
    {
        if
(!httpContext.Request.Headers.TryGetValue(CorrelationIdHeader, out var
correlationId))
        {
            correlationId = Guid.NewGuid().ToString();
            httpContext.Request.Headers[CorrelationIdHeader] =
correlationId;
        }
        httpContext.Response.Headers[CorrelationIdHeader] =
correlationId;
        await _next(httpContext);
    }
}
```

Part 6: Advanced Features in Your Implementation

1. Correlation ID Tracing

Your Implementation:

```
// Middleware adds correlation ID to every request
"AddHeadersToRequest": {
```

```
    "x-correlation-id": "{X-Correlation-Id}"  
  }
```

Flow:

Client Request → API Gateway → Add Correlation ID → Service → Response

2. Authentication per Route

Your Implementation:

```
"AuthenticationOptions": {  
  "AuthenticationProviderKey": "Bearer"  
}
```

Protected Routes:

- /Basket/Checkout - Requires authentication
- /Order - Requires authentication
- /Catalog - No authentication (public)

3. Caching

Your Implementation:

```
.AddCacheManager(x => x.WithDictionaryHandle())
```

Benefits:

- Cache responses to reduce load on services
- Configurable cache duration per route

Part 7: Testing Your API Gateway

Test Routes

```
# Test Catalog service through gateway
curl http://localhost:8010/Catalog/GetAllProducts
```

```
# Test Basket service through gateway
curl http://localhost:8010/Basket/john
```

```
# Test with authentication
curl -H "Authorization: Bearer <token>"
http://localhost:8010/Basket/Checkout
```

Test Rate Limiting (After enabling)

```
# Send 6 requests rapidly
for i in {1..6}; do
    curl http://localhost:8010/Catalog/GetAllProducts
done
```

```
# 6th request should return 429 Too Many Requests
```

Part 8: Your Current API Gateway Features

Feature	Status	Implementation
Request Routing	✓ Active	Routes to all services
Authentication	✓ Active	JWT Bearer token
Correlation ID	✓ Active	Custom middleware
Rate Limiting	✗ Disabled	Configured but commented
Caching	✓ Active	CacheManager

Load Balancing	✓ Active	Multiple host ports
CORS	✓ Active	Configured in Program.cs

Summary

You **ARE** using **API Gateway** with Ocelot in your project:

✓ **Implemented:** Request routing, authentication, correlation ID, caching

✗ **Not Active:** Rate limiting (configured but disabled)

Benefits you're getting:

- Single entry point at <http://localhost:8010>
- Centralized JWT authentication
- Request tracing with correlation IDs
- Response caching
- Clean client URLs

To enable rate limiting: Uncomment the rate limiting configuration in `ocelot.json` and add `.AddPolly()` in `Program.cs`.

Your API Gateway is well-implemented and follows best practices for microservices architecture.